

Migration-Guarded Local Reconfiguration Under Persistent Hotspots in Mobile Edge Systems

Jeng-Shyang Pan^{a,b}, Peng-Yu Duan^a, Li-Fa Liu^a, Qing-Yong Yang^a,
Shu-Chuan Chu^{c,*}

^a*College of Computer Science and Engineering, Shandong University of Science and Technology, Qingdao 266590, China*

^b*Department of Information Management, Chaoyang University of Technology, Taichung 41349, Taiwan*

^c*School of Artificial Intelligence, Nanjing University of Information Science and Technology, Nanjing 210044, China*

Abstract

Persistent local hotspots can make slot-wise placement recomputation incur migration churn. We study whether retaining the previous repaired placement can improve service while limiting reconfiguration. We present **Budget-edLNS**, an implementation-level state-preserving local search that localizes candidates to pressured nodes, bounds the candidate pool, and applies lexicographic acceptance by satisfaction, migration count, and admitted latency. A trial above budget is rejected; any excess introduced by final feasibility repair is recorded, and all methods share that repair. In the implemented simulator, 180 tasks are evaluated for 200 slots across compute, link, and coupled persistent hotspots with 30 paired runs. At the frozen 5% setting, BudgetedLNS improves satisfaction over FullResched by 7.82–8.55 percentage points and reduces the admitted-to-admitted node-change count by 96.39–96.55%; gains over the implemented HIVE comparator are 9.65–13.82 points with 96.74–96.88% fewer such changes. These migration percentages exclude admission and rejection transitions and should not be read as a complete reconfiguration-volume reduction. Sensitivity at 1%, 5%, and 10% reports

*Corresponding author.

Email addresses: 100112@nuist.edu.cn (Jeng-Shyang Pan), 202483060107@sdust.edu.cn (Peng-Yu Duan), 202282060085@sdust.edu.cn (Li-Fa Liu), yang_qy@sdust.edu.cn (Qing-Yong Yang), 100130@nuist.edu.cn (Shu-Chuan Chu)

repair overrides. The evidence is limited to this simulator, load, and hotspot protocol; 5% is not claimed universally optimal.

Keywords: mobile edge computing, service migration, local search

1. Introduction

Mobile edge placement must maintain service while task mobility and infrastructure conditions change. Mobile-edge computing and computation-offloading surveys establish the importance of joint communication and computation decisions [1, 2]. A persistent local compute, communication, or coupled resource degradation is especially difficult because it can make a previously feasible placement unsafe for many consecutive slots. The operational objective is not only high service satisfaction, but also a controlled amount of placement change: migration consumes data-transfer capacity and can itself disrupt service.

A natural response is to solve a new placement problem at every slot. That strategy discards the placement continuity already present in a repaired schedule, and it can generate many more moves than the local disturbance requires. Conversely, never changing the schedule leaves the affected resource overloaded. The unresolved question addressed here is narrow but practical: after a persistent local hotspot appears, how can a controller use the existing schedule while retaining an explicit per-slot local-search budget and exposing any repair-driven violation?

We answer this question with BudgetedLNS. Its central design choice is to treat the prior repaired schedule as the starting point rather than as disposable history. The method detects pressured nodes, ranks risk-exposed tasks, restricts the candidate pool to at most three times the move budget, and accepts a candidate only if it improves a lexicographic objective: satisfaction first, then fewer migrations, then lower admitted latency. A final shared safety repair enforces resource feasibility. The operational contract is explicit: local trials that exceed the move budget are rejected, and a repair-induced excess is retained as a recorded feasibility event rather than silently discarded. The 5% setting is therefore a local-trial budget guard with recorded repair overrides, not a universal absolute migration bound.

The paper makes three bounded contributions.

1. It defines an auditable migration-guarded reconfiguration contract for

persistent hotspots, explicitly separating service performance, migration cost, and repair feasibility.

2. It instantiates a state-preserving BudgetedLNS solver with pressured-node localization, bounded candidate exploration, lexicographic acceptance, and shared feasibility repair.
3. It evaluates the solver in a paired, 30-run, 200-slot protocol on three persistent-hotspot mechanisms. The results establish a service-migration advantage over FullResched and the implemented HIVE comparator only within the stated simulator scope.

2. Related Work and Positioning

Edge and mobile-edge computing research has developed a broad optimization space spanning system architecture, communication, computation, and resource-allocation decisions [3, 1, 2, 4]. This literature motivates dynamic placement, but it does not determine how much of a feasible placement should be retained after a local persistent disturbance.

Mobility-aware placement and migration research explicitly considers where a service should follow a moving user. Survey and scheduling work frames service migration as a joint latency, mobility, container-placement, and resource-management problem [5, 6]. Representative studies investigate dynamic placement [7, 8], proactive microservice placement and migration [9], probabilistic mobility-aware migration [10], and digital-twin-assisted migration [11]. Other systems combine caching with migration [12, 13] or study scalable migration orchestration [14, 15]. BigMEC is particularly relevant because its public implementation exposes a scalable, non-displacing greedy migration rule. We therefore include a code-derived, equal-priority BigMEC greedy adapter as an external reproducible baseline; the adapter is not presented as a reproduction of the complete BigMEC learning system or its San Francisco topology. Our question is narrower: given a current disturbed state and a prior repaired placement, can the controller preserve useful placement continuity under an explicit per-slot local-search budget?

Online MEC controllers also use stability- and learning-oriented formulations for dynamic offloading [16, 17]. Online placement and offloading have been studied with asynchronous decisions [18], two-timescale control [19], and graph-based learning for adaptive microservice placement [20, 21]. More general online reconfiguration has also been analyzed for edge-cloud systems [22]. BudgetedLNS is not presented as a replacement for these controller

families. It instead instantiates large-neighborhood-search principles [23] for a constrained reconfiguration contract: pressure-localized task selection, a candidate pool bounded by the move budget, lexicographic acceptance, and a common feasibility repair. The evaluation therefore compares against slot-wise full rescheduling, the implemented HIVE comparator, and the public-code-derived BigMEC greedy adapter, and it states no cross-topology or deployment generalization.

3. System Model and Objective

At slot t , each of $N = 180$ persistent tasks is either rejected or assigned to one admissible edge resource. Let x_t denote the placement, $s_t(x_t)$ the simulator-defined slot satisfaction score, and $c_t(x_t, x_{t-1})$ the number of tasks whose assignment changes from one nonzero edge node to a different nonzero edge node between slots. Admission of a previously rejected task and rejection of a previously admitted task are service-state transitions, not migrations. The reported **MigrationMB** metric is a profile-derived volume proxy: it sums the simulator’s scaled DNN data-vector footprint, in MB, only for tasks counted by c_t . It is not a packet-level measurement or a complete container-state transfer trace. Resource capacities and communication conditions vary with the scenario, while task mobility is seeded identically across paired methods.

The simulator used five heterogeneous edge nodes. Each task belongs to one of three DNN profiles: VGG16 (20%, base deadline 2.0 s), ResNet50 (50%, 1.0 s), or MobileNetV2 (30%, 0.5 s). At a slot, task deadlines follow $d_i(t) = \max\{0.12, d_i(0)[0.93 + 0.07 \sin(2\pi(t-1)/20 + i/17)]\}$. User positions evolve as clipped Gaussian displacements with standard deviation 35 in a 1000 by 1000 square. Node 1 has compute capacity 350, bandwidth 40 Mb/s, and 8192 MB memory; nodes 2–3 have compute capacity 250, bandwidth 20 Mb/s, and 4096 MB memory; nodes 4–5 have compute capacity 180, bandwidth 40 Mb/s, and 2048 MB memory. These values are the fixed simulator configuration used in all formal cells.

For a placement, the load at node f is $r_f = \sum_i x_{if} \text{Comp}_{if} / \bar{d}$, where $\bar{d}$ is the mean slot deadline. The queue multiplier is $q_f = 1 + r_f / [2(1 - r_f + 10^{-6})]$ for $r_f < 0.99$, and is set to a large penalty otherwise. For an admitted task, latency is $\ell_{if} = \text{Comm}_{if} + \text{Comp}_{if} + \text{Comp}_{if} \max(q_f - 1, 0) + 5 \times 10^{-4} n_f^2$, where n_f is the number of tasks assigned to node f . A task is satisfied only when it has exactly one assignment, its node stays below the memory and load limits, and $\ell_{if} \leq d_i(t)$. The reported satisfaction is $100N^{-1} \sum_i \mathbb{1}[i \text{ is satisfied}]$.

BudgetedLNS uses the local-search budget $B = \max(1, \lceil \rho N \rceil)$. A candidate trial is admissible only when its trial-level migration count is at most B . Here a trial-level migration is a change from one nonzero edge-node assignment to a different nonzero edge-node assignment relative to the previous repaired schedule; admission and rejection transitions are not counted. After the selected local candidate is fixed, the solver applies its final repair and records whether the repaired solver output exceeds B . We denote the recorded excess by $\text{excess} = \max(0, \text{final migrations} - B)$. The evaluation runner then applies the same post-solver feasibility-repair operator to every compared method. BudgetedLNS also invokes this repair inside each local trial and after selecting its local candidate; those calls are part of the solver, whereas the runner call defines the common state used for the reported metrics. The recorded override is computed from that post-runner state relative to the previous repaired assignment. The frozen primary setting uses $\rho = 0.05$, hence $B = 9$. This is a trial-level screening budget, not a hard upper bound on the final repaired schedule. We report mean satisfaction, total node-to-node migrations, the profile-derived `MigrationMB` proxy, mean admitted and rejected task counts, and recorded solver-output repair overrides over a 200-slot horizon. The runner-level repair is common to compared methods so that resource feasibility is not a method-specific measurement artifact.

The historical migration metric is intentionally narrower than a placement-state transition metric. It counts only admitted-to-admitted node changes; admission ($0 \rightarrow f$) and rejection ($f \rightarrow 0$) transitions are excluded from both migration count and `MigrationMB`; they are therefore outside the scope of the historical migration guard as well. The revised runner now exports `AdmissionTransitions`, `RejectionTransitions`, and `TotalPlacementStateTransitions` alongside the legacy fields. Historical summary tables are not silently recomputed because their source CSVs do not retain per-task assignments.

Table 1. Frozen simulator and migration-guard parameters.

Parameter	Value used in the reported experiments
Edge nodes	5 heterogeneous nodes
Tasks	180 persistent tasks: 20% VGG16, 50% ResNet50, 30% MobileNetV2

Parameter	Value used in the reported experiments
Base task deadlines	2.0 s, 1.0 s, and 0.5 s for VGG16, ResNet50, and MobileNetV2
Horizon and replications	200 slots and 30 paired seeds per scenario
Node 1 resources	Compute 350; bandwidth 40 Mb/s; memory 8192 MB
Nodes 2–3 resources	Compute 250; bandwidth 20 Mb/s; memory 4096 MB
Nodes 4–5 resources	Compute 180; bandwidth 40 Mb/s; memory 2048 MB
Mobility model	Clipped Gaussian displacement, standard deviation 35, in a 1000 by 1000 square
Persistent compute hotspot	Node 1 compute capacity at 55% of baseline
Persistent link hotspot	Node 2 bandwidth at 35% of baseline
Persistent coupled hotspot	Node 1 compute at 60%, node 2 bandwidth at 40%, node 2 memory at 70%
Primary BudgetedLNS fraction	$\rho = 0.05$, giving $B = 9$ trial-level moves; final repair may exceed it
Sensitivity fractions	0.01, 0.05, and 0.10
Candidate-pool limit	At most $3B$ risk-ranked tasks
Pressured nodes	Two nodes with the highest pressure score
HIVE controller limits	Five candidate cells, two target cells, and four edits
BigMEC adapter	Equal priority; five-node neighborhood; utility $-(\text{communication} + \text{computation latency})$; non-displacing greedy moves
MPC rollout baseline	Three-step rollout, discount 0.7, two immediate-node candidates

a State-preserving local reconfiguration with a recorded migration guard

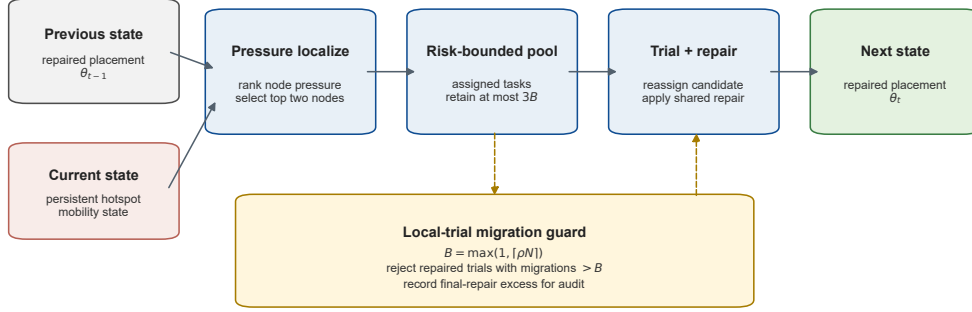


Figure 1: BudgetedLNS preserves a repaired placement and applies a trial-level migration guard to pressure-localized local search. The previous repaired placement and current persistent-hotspot state define node pressure. The solver retains at most $3B$ risk-ranked tasks, rejects repaired local trials above B , and compares admissible candidates lexicographically. Its final repair records any solver-output override and the excess above B ; the evaluation runner applies the same feasibility repair to all compared methods.

Parameter	Value used in the reported experiments
Bootstrap and randomization	20,000 paired bootstrap resamples; 100,000 sign flips; Holm correction

4. Migration-guarded BudgetedLNS

4.1. State-preserving initialization and pressure localization

For slots after initialization, BudgetedLNS begins from the previous slot’s repaired feasible assignment. If no size-compatible previous assignment exists, as in the first slot, the implementation obtains an initial placement from the MPC rollout solver and repairs it before local search. The continuity interpretation therefore applies only after an earlier repaired placement is available. From the current resource state, the method identifies the two most pressured nodes and constructs a risk-ranked destroy set from their assigned tasks. The destroy set focuses search effort on the part of the placement most likely to be invalidated by the persistent hotspot.

4.2. Bounded repair neighborhood

Let $B = \max(1, \text{ceil}(\rho N))$ be the current move budget. BudgetedLNS limits the risk-ranked candidate pool to at most $3B$ tasks and evaluates reassignment actions for each selected task. It repairs every trial before scoring it and discards the trial when the repaired candidate requires more than B migrations relative to the previous assignment. The search stops after the bounded pool is exhausted. This bounded neighborhood prevents a local repair from turning into implicit global rescheduling. The guard is a trial-level screening rule; it does not guarantee that the subsequent feasibility repair will remain below the same count.

4.3. Lexicographic acceptance and shared repair

Candidates are compared lexicographically: higher satisfaction is preferred; ties prefer fewer migrations; remaining ties prefer lower admitted latency. The selected candidate updates the placement, after which BudgetedLNS applies its safety-repair operator once more. The repair checks resource feasibility and handles cases where the local candidate set alone cannot restore a feasible placement. This final solver operation can exceed B ; such an excess is recorded rather than hidden. The runner subsequently applies the same repair to all compared methods for common feasibility evaluation. The 1% sensitivity cells therefore expose the operating trade-off between local-search budget and repair overrides, rather than a universally enforced absolute bound.

4.4. Algorithmic contract

Algorithm 1. BudgetedLNS with repair-aware budget diagnostics

```
Input: state, theta0, budget fraction rho
Output: repaired placement theta and diagnostics
B <- max(1, ceil(rho N)); theta <- theta0
H <- two nodes with highest pressure under theta
P <- risk-ranked tasks assigned to H; keep at most 3B tasks
for each task i in P while moves(theta, theta0) < B do
  best <- theta
  for each candidate resource f do
    trial <- SharedRepair(assign i to f in theta)
    if moves(trial, theta0) <= B and Better(trial, best) then
      best <- trial
    end if
```



```

    end for
    theta <- best
end for
theta <- SharedRepair(theta)
record whether moves(theta, theta0) > B
return theta

```

For reproducibility, the solver follows this order: initialize from the previous repaired placement; compute node pressure; collect assigned tasks on the two most pressured nodes; keep at most $3B$ highest-risk tasks; test reassignment candidates with shared repair; reject trials above B ; apply the lexicographic rule; and finally run shared repair on the accepted placement. The pressure score is the sum of normalized compute load and normalized assigned memory, $p_f = \sum_i x_{if} \text{Comp}_{if} / \bar{d} + m_f / M_f$. The task risk score is $r_i = (\text{Comm}_{if} + \text{Comp}_{if}) / d_i$ on its current node. Exact objective ties retain the incumbent; candidate resources are examined in ascending resource-index order. The primary setting has $\text{rho}=0.05$, $N=180$, and therefore $B=9$. The number of task-resource trials is bounded above by $3B(M-1)$ before early stopping, where M is the number of edge resources. This description distinguishes the method’s budget-guarded local-search contract from generic unconstrained LNS.

5. Experimental Protocol

5.1. Frozen paired evaluation

The primary experiment uses 30 independent paired runs for each of three persistent-hotspot scenarios: compute, link, and coupled compute/link/memory. Each run has 200 slots and $N=180$ tasks. All methods receive identical task instances, mobility perturbations, hotspot states, final repair, metric implementation, and scenario/run seed. The primary comparison is BudgetedLNS versus FullResched. HIVE is evaluated only as an internally implemented hierarchical comparator, and the code-derived, equal-priority, non-displacing BigMEC greedy adapter is an external reproducible baseline. All four methods use identical formal cells. In every slot, compute hotspots reduce node 1 compute capacity to 55% of baseline; link hotspots reduce node 2 bandwidth to 35%; and coupled hotspots reduce node 1 compute capacity to 60%, node 2 bandwidth to 40%, and node 2 memory to 70%. The degradation begins at the first slot and remains active throughout the 200-slot horizon.

The frozen acceptance criterion requires satisfaction non-inferiority (95% confidence-interval lower bound above -1 percentage point), migration reduction of at least 30%, and a Holm-adjusted sign-flip $p < 0.05$ for the paired migration difference in at least two scenarios. The manuscript reports the criterion as a formal evaluation rule, not as a preregistration claim. The number and magnitude of any final-repair override are reported rather than presuming that the 5% guard is not an absolute bound.

5.2. *Baselines, pairing, and reporting*

FullResched recomputes a placement independently at each slot with the implemented MPC rollout solver. It orders tasks by deadline, examines the two best immediate nodes, uses a three-step rollout with discount 0.7, and penalizes memory, overload, and deadline violations before the common final repair. The HIVE comparator is the implemented hierarchical controller: it discovers fragility cells, estimates propagation, plans interventions, realizes sparse micro-edits, and runs a three-step counterfactual stabilization check. Its default controller limits are five candidate cells, two target cells, and four edits. The BigMEC adapter maps a service to a persistent DNN task, assigns unit priority, treats the five nodes as the destination neighborhood, and maximizes negative communication-plus-computation latency subject to DNN-memory capacity. It follows the public non-displacing greedy contract but then receives the same final Safe Harbor repair as every other method. It does not execute the original BigMEC learning components, 50-node topology, or trace. The adapter was audited against the GPL-3.0 repository at <https://github.com/flbrandh/MEC-Simulator-2-BigMEC>, commit `a586ae3b98119f83bb7883bd94ea95b1fbed7a8c`, and the archived software DOI <https://doi.org/10.5281/zenodo.10810301>. Within each scenario, a paired run uses the same task instance, mobility sequence, persistent degradation trace, final repair, and metric implementation for every method. The reported paired confidence intervals and Holm-adjusted sign-flip tests therefore evaluate within-seed method differences. This evaluation does not claim a comparison with every controller class, a real deployment, or generalization across topology and load. The BudgetedLNS candidate-evaluation count and the FullResched MPC rollout depth are implementation-specific and are not matched by a common wall-clock or candidate-evaluation budget. The comparison is therefore an outcome comparison under the frozen implementations, not a claim of computational-efficiency superiority.

5.3. Hotspot onset and component ablation

The hotspot-onset experiment first runs 50 normal slots to form a feasible, method-specific deployment. At slot 51, the designated compute, link, or coupled degradation is applied and remains active through slot 200. We report trajectories and post-onset outcomes so that adaptation is not confounded with first-slot initialization.

The component experiment uses the same slot-51 onset protocol and changes one design choice at a time relative to the frozen 5% method. The first variant replaces state-preserving initialization with a fresh MPC rollout at every slot. The second replaces the pressure-localized, at-most- $3B$ pool with all currently admitted tasks. The third disables the 5% local trial guard while retaining state preservation and pressure localization. All variants retain the common feasibility repair, inputs, seeds, and metrics. This isolates the operational contribution of initialization, candidate localization, and guarding within the tested simulator.

Table 2. Frozen formal evaluation protocol.

Protocol item	Frozen primary configuration
Task population and horizon	180 persistent tasks; 200 slots per run
Scenarios	Compute, link, and coupled compute/link/memory persistent hotspots
Replication	30 independent paired seeds per scenario
Primary budget	$\rho = 0.05$, $B = 9$ trial-level local-search migrations; final repair overrides are recorded
Shared components	Inputs, mobility, hotspot trace, final repair, and metric implementation
Hotspot-onset cell	Slots 1–50 normal; hotspot introduced at slot 51 and maintained through slot 200
Component variants	Cold start; full admitted-task candidate pool; no local-trial guard
Compared methods	FullResched; implemented HIVE comparator; code-derived BigMEC greedy adapter; BudgetedLNS

Protocol item	Frozen primary configuration
Statistical reporting	Stored paired 95% confidence intervals and Holm-adjusted sign-flip tests

5.4. Statistical analysis and runtime provenance

For each scenario, the paired satisfaction effect is the mean of the 30 within-seed differences. Its 95% interval is a percentile bootstrap with 20,000 resamples and fixed Twister seeds recorded in the analysis scripts. The migration test uses a two-sided paired sign-flip randomization test with 100,000 sampled sign patterns and fixed seed specified in the analysis script for 30 pairs, followed by Holm adjustment within each baseline’s three scenario-wise tests. The CSV files retain MATLAB wall-clock measurements around each solver call, excluding the runner’s common final repair. Because independent experiment shards ran concurrently, absolute runtime is retained for provenance but is not used for a comparative performance claim or interpreted as deployment latency. Experiments used MATLAB R2025a on Windows 11 (Intel Core Ultra 7 155H, 16 physical cores, 22 logical processors, 31.61 GiB RAM). Analysis and figures used Python 3.13.7, NumPy 2.3.3, pandas 2.3.2, and matplotlib 3.10.6.

Before analysis, the pipeline checks every slot-level row for every scenario, method, and seed. A row passes only when `AdmittedTasks + RejectedTasks = N`, the resource-violation rate is zero, and the method-run contains each slot 1–200 exactly once. Run-level admission and rejection means are then recomputed from those 200 rows and matched against the stored run summary. Admission or rejection transitions remain excluded from both migration count and `MigrationMB`.

6. Results

6.1. Main paired result

BudgetedLNS satisfies the frozen acceptance criterion in all three scenarios within the stated simulator and protocol. Its satisfaction improvement over FullResched is consistently positive, and its reduction in the admitted-to-admitted node-change count is large. This result does not establish a comparable reduction in complete placement-state transitions, because the historical formal tables do not retain the per-task assignments needed to

reconstruct admission and rejection transitions. Figure 2 shows the run-level outcomes; Table 3 reports the method-level service, migration, and admission means, followed by the paired effect estimates from the frozen formal analysis.

Table 3: Formal 30-run means at the 5% primary guard. “Admitted” and “Rejected” are mean task counts per slot (and sum to $N = 180$). MigrationMB is the profile-derived migration-volume proxy defined in Section 3; it is not packet-level network traffic. Admission and rejection transitions are excluded from migration counts.

Scenario	Method	Satisfaction (%)	Migrations	MigrationMB	Admitted	Rejected
Compute	FullResched	53.12	9733.8	14215.44	139.57	40.43
Compute	HIVE comparator	47.85	10784.1	15234.26	139.71	40.29
Compute	BigMEC adapter	43.24	4131.4	6059.78	139.73	40.27
Compute	BudgetedLNS	61.67	336.2	527.90	139.21	40.79
Link	FullResched	45.11	8897.8	19415.58	153.21	26.79
Link	HIVE comparator	43.28	10248.2	20072.28	153.36	26.64
Link	BigMEC adapter	37.43	4207.9	9403.81	153.42	26.58
Link	BudgetedLNS	52.93	321.4	2055.00	152.89	27.11
Coupled	FullResched	51.77	9620.2	14154.74	141.04	38.96
Coupled	HIVE comparator	48.89	10349.7	14576.48	141.16	38.84
Coupled	BigMEC adapter	40.24	3831.8	5595.98	141.28	38.72
Coupled	BudgetedLNS	60.16	337.0	510.65	140.90	39.10

The main table reports the service and migration outcomes. Here, “migration” means only an admitted-to-admitted node change; the table does not report a complete placement-state transition count. Because the trial-level guard and the final repair have different contracts, we report their audit separately in Table 4; this prevents a repair-induced excess from being mistaken for a silently enforced budget.

Table 4: Formal audit of BudgetedLNS trial-level guard and final-repair overrides (30 runs per persistent-hotspot scenario, $N = 180$, 200 slots, $\rho = 0.05$, $B = 9$). “Repair overrides/run” is the mean number of recorded final-repair override events per 200-slot run; “maximum excess” is the largest recorded $\max(0, \text{final migrations} - B)$ across the 30 runs.

Scenario	B	Repair overrides/run	Maximum excess
Compute hotspot	9	0.90	3
Link hotspot	9	0.60	6
Coupled hotspot	9	1.33	5

Paired BudgetedLNS versus FullResched results at the 5% primary setting.

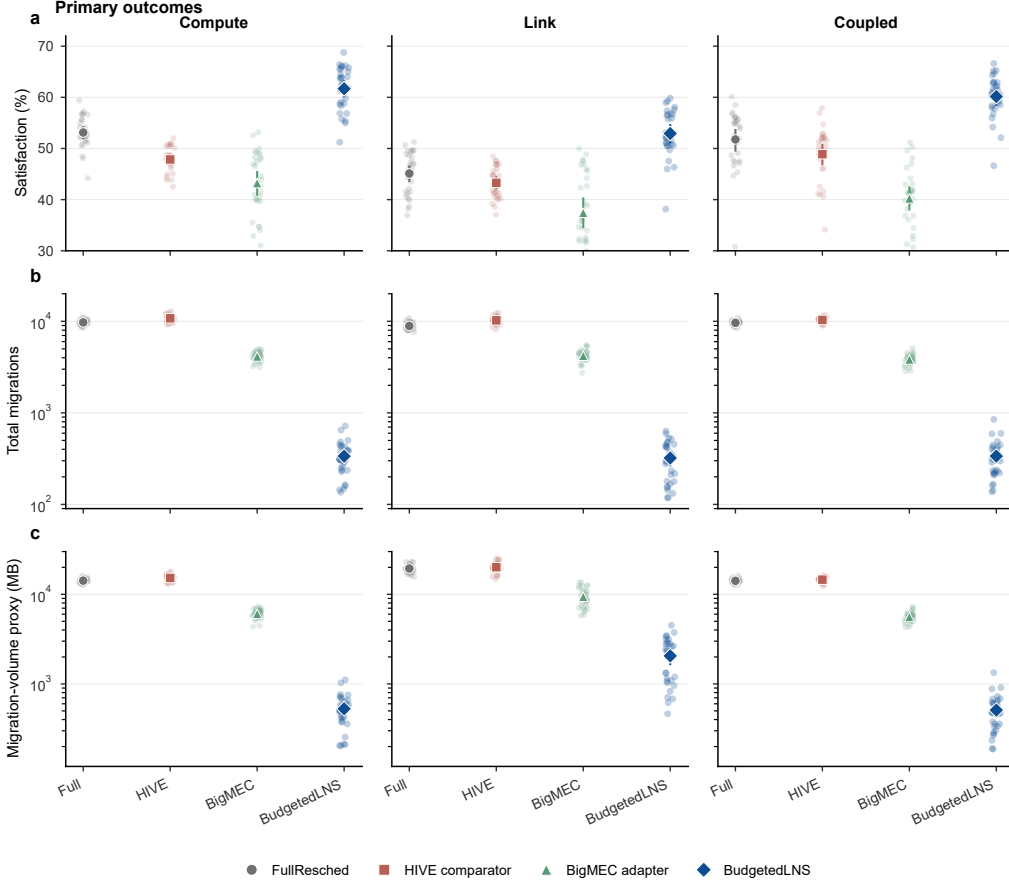


Figure 2: BudgetedLNS separates from the comparator methods in run-level service and migration outcomes. Columns show compute, link, and coupled persistent hotspots. Rows show (a) mean satisfaction, (b) total migrations on a logarithmic scale, and (c) the profile-derived migration-volume proxy. Faint points are individual formal runs; diamonds and whiskers are arithmetic means and deterministic 95% bootstrap intervals. Each method-scenario cell contains 30 matched 200-slot runs at $N = 180$ and the 5% primary local-search guard. MigrationMB is a profile-derived volume proxy, not packet-level network traffic.

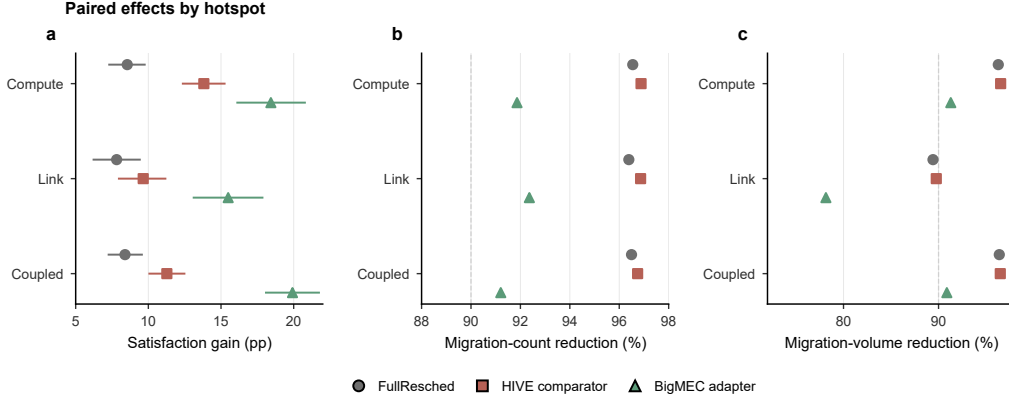


Figure 3: Paired effect sizes from the frozen formal experiment. (a) Satisfaction gain over each comparator. (b) Migration-count reduction. (c) Migration-volume-proxy reduction. Each scenario has 30 paired 200-slot runs at $N = 180$; markers identify the implemented comparator and whiskers show the stored paired interval where available. Runtime is retained only as provenance because experiment shards executed concurrently.

Scenario	LNS - Full satisfaction (pp)	95% CI (pp)	Migration reduction	Holm-adjusted p (migration)
Compute hotspot	+8.55	[7.31, 9.75]	96.55%	0.000030
Link hotspot	+7.82	[6.23, 9.41]	96.39%	0.000030
Coupled hotspot	+8.39	[7.27, 9.55]	96.50%	0.000030

6.2. Implemented hierarchical comparator

The matched HIVE comparator exhibits lower mean satisfaction and substantially higher migration counts in every tested persistent-hotspot cell. BudgetedLNS has a 9.65–13.82-point satisfaction advantage and 96.74–96.88% fewer admitted-to-admitted node changes (Figure 3). Against the code-derived BigMEC adapter, the corresponding advantages are 15.49–19.92 points and 91.21–92.36% fewer admitted-to-admitted node changes. These are bounded comparator comparisons, without extending the claims to the complete BigMEC system or to untested hierarchical or learned controllers.

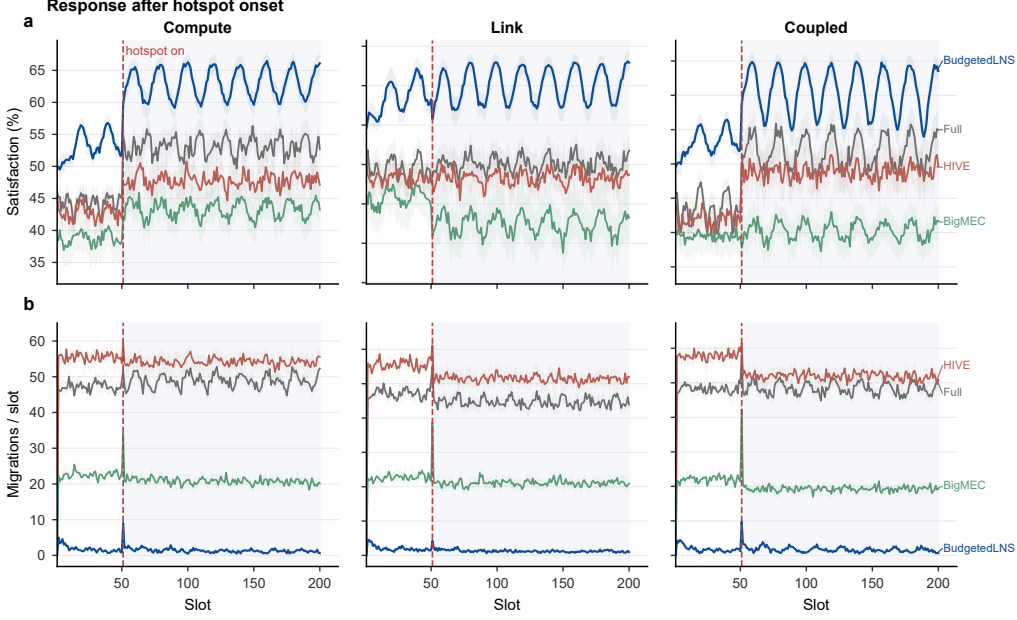


Figure 4: Response to a persistent hotspot introduced after a feasible warm-up. The dashed line marks slot 51, when the hotspot is imposed and then held fixed. (a) Slot-wise satisfaction and (b) slot-wise migration count, with means and 95% bootstrap bands across 30 paired runs per scenario. Admission and rejection transitions are excluded from both migration measures.

6.3. Hotspot onset and persistence

The onset experiment starts from a normal, feasible deployment for slots 1–50 and introduces the selected compute, link, or coupled hotspot at slot 51, after which the condition is maintained through slot 200. The slot-level trajectories in Figure 4 therefore show the post-onset response rather than a cold-start stress test. Across the three onset cells, BudgetedLNS improves post-onset satisfaction over FullResched by 8.77–9.95 percentage points and reduces post-onset migrations by 96.49–97.18% in the admitted-to-admitted node-change count. The corresponding HIVE-comparator gains are 11.71–15.21 points and 96.73–97.52% fewer admitted-to-admitted node changes. These paired effects use the same 30 onset seeds and should be interpreted only for the specified slot-51 intervention.

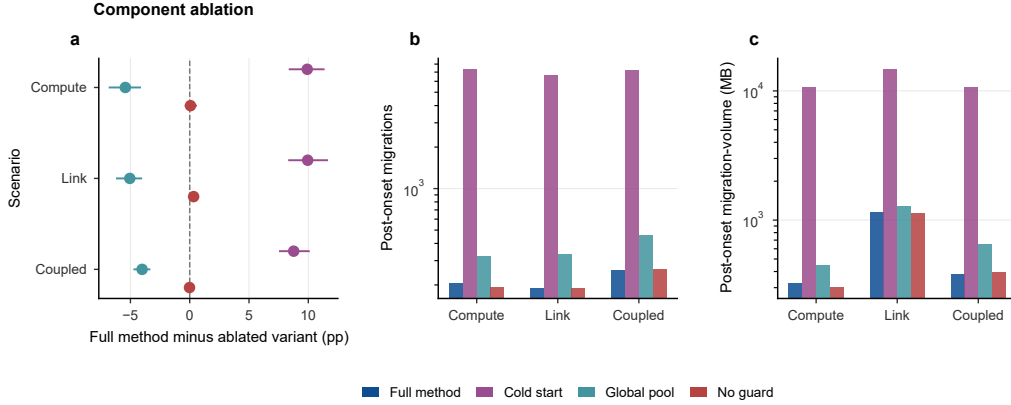


Figure 5: Minimum component ablation under the same persistent-hotspot protocol. (a) Full method minus variant post-onset satisfaction. (b) Post-onset migration count and (c) migration-volume proxy. Points and whiskers in (a) are paired means and 95% bootstrap intervals; bars in (b,c) are run means across 30 seeds per scenario.

6.4. Component ablation

The minimum component ablation keeps the common inputs, repair, and metrics fixed while changing one operational component at a time. Removing state preservation (cold start) lowers post-onset satisfaction by 8.77–9.95 points and increases post-onset migrations from 191–255 to 6,710–7,336 per run. Replacing the pressure-localized pool with a global candidate pool lowers satisfaction by 4.02–5.43 points. Removing the trial-level budget guard changes satisfaction by less than 0.33 points in absolute value in these cells, but removes the guard’s recorded budget diagnostic. Figure 5 reports these paired effects and the corresponding post-onset migration volumes; the results support the operational role of state continuity and candidate localization without claiming a universal causal decomposition.

6.5. Bounded-search diagnostics

The frozen 5% replay evaluates approximately 21.35k–21.49k candidates per 200-slot run (106.75–107.46 evaluations per slot) and records 255.5–307.8 accepted reconfiguration moves. The 5% local-search trial guard is active in only 1.20–3.17 of 200 slots on average (Figure 6). Thus, the observed savings do not arise from saturating the local-search budget at every slot. Because admission and rejection remain scenario-dependent, this diagnostic

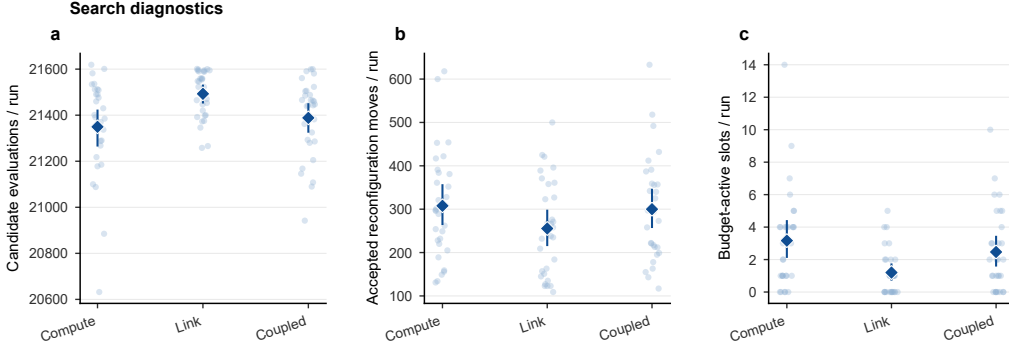


Figure 6: BudgetedLNS uses a bounded search neighborhood and rarely reaches its 5% local-search trial guard. (a) Candidate evaluations per slot. (b) Recorded accepted reconfiguration moves per 200-slot run. (c) Budget-active slots per 200-slot run. Boxplots and faint points show run-level distributions; diamonds and whiskers are arithmetic means and deterministic 95% bootstrap intervals over 30 frozen runs per scenario. These diagnostics describe observed search behavior and do not identify a causal decomposition.

supports the bounded-search interpretation but is not a causal decomposition of satisfaction gains.

6.6. Migration-budget boundary

The 1%, 5%, and 10% sensitivity study isolates the budget parameter. At 1%, the local-search guard permits only two moves per slot and the final repair records frequent overrides (15.97–26.17 per run on average, with maximum excesses of 6–7 in the completed sensitivity cells). The 5% primary cell still records a small number of overrides (0.60–1.33 per run; maximum excess 3–6), while the 10% cell is nearly override-free (0–0.10 per run; maximum excess 0–1). These are operating-point diagnostics, not evidence of a universal absolute bound (Figure 7).

7. Discussion and Limitations

The observed advantage is consistent with placement continuity being useful when the disturbance is persistent and spatially localized. Starting from a repaired schedule permits the controller to direct changes toward pressured resources instead of recreating unrelated placement churn. However, the present diagnostics do not isolate the causal contribution of state preservation, pressure localization, and the 3B pool bound. An alternative explanation

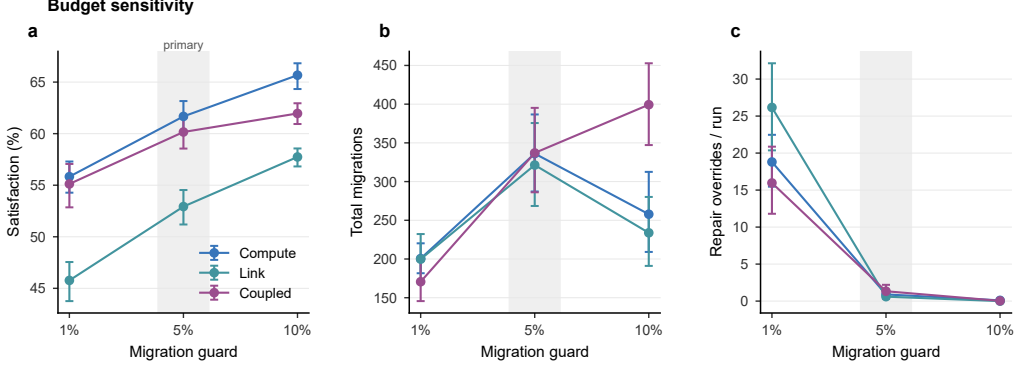


Figure 7: The migration guard exhibits a budget–feasibility trade-off. (a) Run-level satisfaction, (b) total migrations, and (c) recorded repair overrides by guard fraction. Lines and markers are means across 30 runs per scenario; overrides are retained as observed solver-output events and are not relabeled as budget-excess events.

is that part of the apparent benefit reflects interactions with the chosen feasibility repair. The common final repair and matched input traces reduce this concern, but they do not establish equivalence to all alternative repair policies. The method should therefore be understood as a reproducible engineering composition of known search and repair operations, rather than as a new optimization theory or a proof of global optimality. The result is operationally useful only when the trial-level guard and final repair can accommodate feasibility recovery; the 1% boundary makes that condition explicit, and the final schedule is not guaranteed to satisfy the nominal budget.

The fixed implementations also leave computational-budget fairness open: BudgetedLNS explores a bounded candidate pool, whereas FullResched uses a three-step MPC rollout. Runtime was retained for provenance and is not used as evidence that either implementation is computationally superior.

The study is limited to one implemented simulator, $N=180$, three persistent hotspot mechanisms, and tested budgets. It does not validate changing loads, alternative topologies, arbitrary disturbance durations, or live deployment. The historical formal CSV files report admitted-to-admitted node migrations but do not retain per-task assignments; therefore admission and rejection transitions cannot be retrospectively reconstructed from those summaries. The revised runner exports these transitions for future formal reruns, but they are not silently substituted into the existing main tables. These extensions and the complete transition audit require newly designed or rerun experiments

and are outside this submission’s quantitative claims.

8. Conclusion

This paper studies reconfiguration after persistent edge hotspots under an explicit local-search migration budget. BudgetedLNS preserves the previous repaired schedule, localizes the search to pressured resources, and uses lexicographic acceptance with shared feasibility repair. In the tested paired simulator protocol, it improves satisfaction by 7.82–8.55 percentage points over slot-wise FullResched and by 9.65–13.82 points over the implemented HIVE comparator, while reducing the admitted-to-admitted node-change count by 96.39–96.88%. Admission and rejection transitions are excluded from this percentage. A 1% guard often requires repair overrides, whereas the frozen 5% setting substantially reduces but does not eliminate them. Extending this evidence to new scales, topologies, repair policies, component-isolation studies, and deployments is a concrete next step rather than a claim of this paper.

CRedit authorship contribution statement

Author-specific CRediT roles must be completed and approved by all authors before submission.

Funding

Funding information, or an author-approved no-funding statement, must be confirmed before submission.

Declaration of competing interest

The authors must confirm the final declaration before submission.

Data availability

The code and simulator-generated data are publicly available in the GitHub repository *Migration-Guarded-Local-Reconfiguration-Under-Persistent-Hotspots-in-Mobile-Edge-Systems*. The dataset and associated code package are entitled *MGR-PH* (full title: *Migration-Guarded Local Reconfiguration Under Persistent Hotspots in Mobile Edge Systems*). The

repository contains the MATLAB simulator and experiment code, frozen run-level and slot-level data, figure source data, manuscript sources, and checksum records. No separate external dataset was used.

References

- [1] P. Mach, Z. Becvar, Mobile edge computing: A survey on architecture and computation offloading, *IEEE Communications Surveys & Tutorials* 19 (3) (2017) 1628–1656. doi:10.1109/COMST.2017.2682318.
- [2] Y. Mao, C. You, J. Zhang, K. Huang, K. B. Letaief, A survey on mobile edge computing: The communication perspective, *IEEE Communications Surveys & Tutorials* 19 (4) (2017) 2322–2358. doi:10.1109/COMST.2017.2745201.
- [3] W. Shi, J. Cao, Q. Zhang, Y. Li, L. Xu, Edge computing: Vision and challenges, *IEEE Internet of Things Journal* 3 (5) (2016) 637–646. doi:10.1109/JIOT.2016.2579198.
- [4] N. Abbas, Y. Zhang, A. Taherkordi, T. Skeie, Mobile edge computing: A survey, *IEEE Internet of Things Journal* 5 (1) (2018) 450–465. doi:10.1109/JIOT.2017.2750180.
- [5] S. Wang, J. Xu, N. Zhang, Y. Liu, A survey on service migration in mobile edge computing, *IEEE Access* 6 (2018) 23511–23528. doi:10.1109/ACCESS.2018.2828102.
- [6] O. Oleghe, Container placement and migration in edge computing: Concept and scheduling models, *IEEE Access* 9 (2021) 68028–68043. doi:10.1109/ACCESS.2021.3077550.
- [7] T. Ouyang, Z. Zhou, X. Chen, Follow me at the edge: Mobility-aware dynamic service placement for mobile edge computing, *IEEE Journal on Selected Areas in Communications* 36 (10) (2018) 2333–2345. doi:10.1109/JSAC.2018.2869954.
- [8] X. Zhao, Y. Shi, S. Chen, Maesp: Mobility aware edge service placement in mobile edge networks, *Computer Networks* 182 (2020) 107435. doi:10.1016/j.comnet.2020.107435.

- [9] K. Ray, A. Banerjee, N. C. Narendra, Proactive microservice placement and migration for mobile edge computing, in: 2020 IEEE/ACM Symposium on Edge Computing, 2020, pp. 28–41. doi:10.1109/SEC50012.2020.00010.
- [10] M. Xu, Q. Zhou, H. Wu, W. Lin, K. Ye, C. Xu, Pdma: Probabilistic service migration approach for delay-aware and mobility-aware mobile edge computing, *Software: Practice and Experience* 52 (2) (2021) 394–414. doi:10.1002/SPE.3014.
- [11] E. Bozkaya, Digital twin-assisted and mobility-aware service migration in mobile edge computing, *Computer Networks* 231 (2023) 109798. doi:10.1016/J.COMNET.2023.109798.
- [12] C. Li, L. Zhu, W. Li, Y. Luo, Joint edge caching and dynamic service migration in sdn based mobile edge computing, *Journal of Network and Computer Applications* 177 (2021) 102966. doi:10.1016/J.JNCA.2020.102966.
- [13] C. Li, Y. Zhang, X. Gao, Y. Luo, Energy-latency tradeoffs for edge caching and dynamic service migration based on dqn in mobile edge computing, *Journal of Parallel and Distributed Computing* 166 (2022) 15–31. doi:10.1016/J.JPDC.2022.03.001.
- [14] F. Brandherm, J. Gedeon, O. Abboud, M. Muhlhauser, Bigmec: Scalable service migration for mobile edge computing, in: 2022 IEEE/ACM 7th Symposium on Edge Computing, 2022, pp. 136–148. doi:10.1109/SEC54971.2022.00018.
- [15] flbrandh, flbrandh/mec-simulator-2-bigmec: v1.0, Zenodo software archive (2024). doi:10.5281/zenodo.10810301.
URL <https://doi.org/10.5281/zenodo.10810301>
- [16] Y. Mao, J. Zhang, K. B. Letaief, Dynamic computation offloading for mobile-edge computing with energy harvesting devices, *IEEE Journal on Selected Areas in Communications* 34 (12) (2016) 3590–3605. doi:10.1109/JSAC.2016.2611964.
- [17] S. Bi, L. Huang, H. Wang, Y.-J. A. Zhang, Lyapunov-guided deep reinforcement learning for stable online computation offloading in mobile-edge

- computing networks, *IEEE Transactions on Wireless Communications* 20 (11) (2021) 7519–7537. doi:10.1109/TWC.2021.3085319.
- [18] X. Li, X. Zhang, T. Huang, Asynchronous online service placement and task offloading for mobile edge computing, in: 2021 18th Annual IEEE International Conference on Sensing, Communication, and Networking, 2021, pp. 1–9. doi:10.1109/SECON52354.2021.9491595.
 - [19] X. Li, X. Zhang, T. Huang, Joint task offloading and service placement for mobile edge computing: An online two-timescale approach, *IEEE Transactions on Cloud Computing* 11 (4) (2023) 3656–3671. doi:10.1109/TCC.2023.3312283.
 - [20] L. Chen, Y. Bai, P. Zhou, Y. Li, Z. Qu, J. Xu, On adaptive edge microservice placement: A reinforcement learning approach endowed with graph comprehension, *IEEE Transactions on Mobile Computing* 23 (12) (2024) 11144–11158. doi:10.1109/TMC.2024.3396510.
 - [21] K. Ray, A. Banerjee, N. C. Narendra, Learning-based microservice placement and migration for multi-access edge computing, *IEEE Transactions on Network and Service Management* 21 (2) (2024) 1969–1982. doi:10.1109/TNSM.2023.3344192.
 - [22] I.-H. Hou, T. Zhao, S. Wang, K. Chan, Asymptotically optimal algorithm for online reconfiguration of edge-clouds, in: Proceedings of the 17th ACM International Symposium on Mobile Ad Hoc Networking and Computing, 2016, pp. 291–300. doi:10.1145/2942358.2942363.
 - [23] D. Pisinger, S. Ropke, Large neighborhood search, in: Handbook of Metaheuristics, Springer US, 2010, pp. 399–419. doi:10.1007/978-1-4419-1665-5_13.